

Table of Contents

1. Problem Statement.....	1
2. Dataset Description.....	2
3. Summary of Data Analysis.....	3
4. HDFS Analysis	4
5. MapReduce Analysis	10
6. Pig Analysis.....	18
7. Result.....	22
8. Conclusion	25

YouTube Video Data Analysis Using Hadoop HDFS, MapReduce and Apache Pig

1. Problem Statement

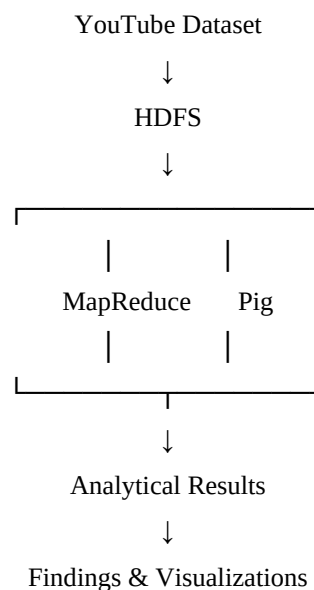
The rapid growth of online video platforms has created a large amount of data about videos, uploaders, categories, views, ratings, and comments. Processing such large datasets requires efficient storage and processing techniques.

This project analyzes a large YouTube dataset using HDFS, Java MapReduce, and Apache Pig. The main analyses are:

1. Total number of YouTube video records.
2. Number of videos in each category.
3. Total views by category.
4. Average rating by category.

The project also compares the results from MapReduce and Pig to verify the accuracy of the analysis and demonstrate different Big Data processing approaches.

The overall workflow of the project is:



Thus, the project demonstrates how a large dataset can be stored, processed, aggregated, and analyzed using Hadoop-based Big Data technologies.

2. Dataset Description

2.1 Source

The dataset used in this project is the YouTube Dataset, which contains information about YouTube videos.

The reference material identifies the original dataset source as:

YouTube Dataset:

<http://netsg.cs.sfu.ca/youtubedata/>

The dataset contains information such as video ID, uploader, age, category, length, views, rating, number of ratings, comments, and related video IDs.

For this project, the dataset was divided into five input files:

0.txt

1.txt

2.txt

3.txt

4.txt

Together, these files contain: 749,361 records

The raw dataset was used locally for preparation and was then uploaded to HDFS for Big Data processing.

2.2 Fields

The dataset contains the following major fields:

Field	Description
video_id	Unique identifier of a YouTube video
uploader	Username of the video uploader
age	Number of days between video upload date and February 15, 2007
category	Category selected for the video
length	Length of the video
views	Number of views received by the video
rate	Rating of the video
ratings	Number of ratings received
comments	Number of comments received
Related IDs	Related YouTube video IDs

3. Summary of Data Analysis

3.1 Key Analytical Steps

The analysis was performed in several stages.

Step 1: Data Storage

The five YouTube dataset files were uploaded to HDFS.

The HDFS input directory was: `/bda_final_project/input`

The uploaded files were checked using Hadoop commands.

Step 2: HDFS Verification

After uploading the dataset, HDFS was checked to verify:

- Files were successfully uploaded.
- HDFS blocks were created.
- DataNode was active.
- No blocks were missing.
- No blocks were corrupt.
- The stored dataset was accessible.

Step 3: Total Record Analysis

The total number of records was calculated using Apache Pig.

The result was: 749,361 records

This result was also verified through HDFS data counting.

Step 4: Category-wise Video Count

The dataset was grouped according to the `category` field.

The number of videos in each category was then calculated.

This represents a data summarization and grouping operation.

The result showed that: Music = 179,049 videos

which was the largest category.

Step 5: Category-wise Total Views

The records were grouped by category and the `views` field was summed.

This analysis identifies which category accumulated the largest total number of views.

The result was: Music = 2,426,199,511 views

Step 6: Category-wise Average Rating

The records were grouped according to category and the average of the `rate` field was calculated.

The highest average rating was: Film & Animation ≈ 4.08

Step 7: Cross-Technology Verification

The category-wise analyses were implemented independently in:

- Java MapReduce

- Apache Pig

The results were compared.

The consistency between the two approaches provides evidence that the aggregation logic was correctly implemented.

3.2 Outputs

The project produced four major analytical outputs:

Output 1: Total Records

749,361

Output 2: Category-wise Video Count

Music → 179,049

Output 3: Category-wise Total Views

Music → 2,426,199,511

Output 4: Category-wise Average Rating

Film & Animation → 4.0777 approximately

The reference project criteria similarly include category occurrence, total views, rating-based analysis, and summarization as important analytical tasks.

4. HDFS Analysis

Hadoop Distributed File System (HDFS) was used as the main storage layer for the YouTube Big Data Analysis project. HDFS provides a Hadoop-based file system for storing and accessing large datasets. In this project, the YouTube dataset was stored in HDFS before performing analysis using MapReduce and Apache Pig.

The Hadoop environment used for the project was:

Component	Version
Hadoop	3.2.4
Java	1.8.0_411
HDFS Project Directory	/bda_final_project

These versions were used to configure and execute the Hadoop environment for the project.

4.1 HDFS Directory Structure

A separate project directory was created in HDFS: /bda_final_project

Two subdirectories were created:

/bda_final_project/input

/bda_final_project/output

The input directory was used to store the original YouTube dataset files, while the output directory was reserved for Hadoop processing results.

The following commands were used:

```
hdfs dfs -mkdir -p /bda_final_project/input
hdfs dfs -mkdir -p /bda_final_project/output
```

These commands created the required HDFS directory structure for the project.

4.2 Uploading Dataset into HDFS

The YouTube dataset consisted of five text files:

0.txt
1.txt
2.txt
3.txt
4.txt

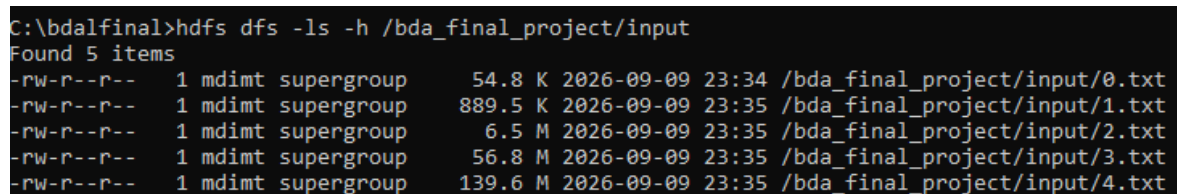
Each file was uploaded individually from the local dataset directory into the HDFS input directory.

The commands used were:

```
hdfs dfs -put C:\bdalfinal\bigdataset\0.txt /bda_final_project/input/
hdfs dfs -put C:\bdalfinal\bigdataset\1.txt /bda_final_project/input/
hdfs dfs -put C:\bdalfinal\bigdataset\2.txt /bda_final_project/input/
hdfs dfs -put C:\bdalfinal\bigdataset\3.txt /bda_final_project/input/
hdfs dfs -put C:\bdalfinal\bigdataset\4.txt /bda_final_project/input/
```

Therefore, the local YouTube dataset was successfully transferred into HDFS for distributed processing.

Screenshot



```
C:\bdalfinal>hdfs dfs -ls -h /bda_final_project/input
Found 5 items
-rw-r--r--  1 mdimt supergroup    54.8 K 2026-09-09 23:34 /bda_final_project/input/0.txt
-rw-r--r--  1 mdimt supergroup   889.5 K 2026-09-09 23:35 /bda_final_project/input/1.txt
-rw-r--r--  1 mdimt supergroup    6.5 M 2026-09-09 23:35 /bda_final_project/input/2.txt
-rw-r--r--  1 mdimt supergroup    56.8 M 2026-09-09 23:35 /bda_final_project/input/3.txt
-rw-r--r--  1 mdimt supergroup   139.6 M 2026-09-09 23:35 /bda_final_project/input/4.txt
```

Fig1: hdfs_upload.png.

Description

The screenshot shows the successful uploading of the five YouTube dataset files into the HDFS input directory.

4.3 HDFS Block Analysis

One of the important features of HDFS is that large files are divided into blocks for storage and processing.

The block information for the project dataset was checked using:

hdfs fsck /bda_final_project/input -files -blocks -locations

This command provides information about:

- HDFS files
- Number of blocks
- Block locations
- Replication
- File-system health

The analysis showed that the five input files occupied a total of: 6 HDFS blocks

The distribution was:

File	HDFS Blocks
0.txt	1
1.txt	1
2.txt	1
3.txt	1
4.txt	2
Total	6

The largest file, 4.txt, required two HDFS blocks, while each of the other four files occupied one block. This demonstrates how HDFS manages files by dividing them into blocks rather than treating the entire dataset as one storage unit.

Screenshot

```
/bda_final_project/input/4.txt 146345957 bytes, replicated: replication=1, 2 block(s): OK
0. BP-463196398-192.168.1.105-1785090964085:blk_1073741909_1085 len=134217728 Live_repl=1 [DatanodeInfoWithStorage[127.0.0.1:9866,DS-93d006e2-c07f-4021-a5ff-48a9065ea91d,DISK]]
1. BP-463196398-192.168.1.105-1785090964085:blk_1073741910_1086 len=12128229 Live_repl=1 [DatanodeInfoWithStorage[127.0.0.1:9866,DS-93d006e2-c07f-4021-a5ff-48a9065ea91d,DISK]]

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 213620517 B
Total files: 5
Total blocks (validated): 6 (avg. block size 35603419 B)
Minimally replicated blocks: 6 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
```

Fig2: HDFS file, block by the hdfs fsck command.

```

Number of data-nodes: 1
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 213620517 B
Total files: 5
Total blocks (validated): 6 (avg. block size 35603419 B)
Minimally replicated blocks: 6 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
FSCK ended at Wed Sep 09 23:58:52 BDT 2026 in 13 milliseconds

The filesystem under path '/bda_final_project/input' is HEALTHY
C:\bdalfinal>_

```

Fig3: HDFS block-location information generated by the hdfs fsck command.

Description: The screenshots show the HDFS file, block, and block-location information generated by the hdfs fsck command.

4.4 DataNode Analysis

The Hadoop DataNode status was checked using:

```
hdfs dfsadmin -report
```

This command provides information about the available DataNodes and their storage capacity.

The project environment successfully reported an active DataNode.

The DataNode report was used to verify that Hadoop had an active storage node available for storing and processing the project dataset.

Screenshot:

```
C:\bdalfinal>hdfs dfsadmin -report
Configured Capacity: 127445282816 (118.69 GB)
Present Capacity: 32116406321 (29.91 GB)
DFS Remaining: 31842705408 (29.66 GB)
DFS Used: 273700913 (261.02 MB)
DFS Used%: 0.85%
Replicated Blocks:
    Under replicated blocks: 10
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 10
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0

-----
Live datanodes (1):

Name: 127.0.0.1:9866 (127.0.0.1)
Hostname: DESKTOP-MJE7U90.netis.cc
Decommission Status : Normal
Configured Capacity: 127445282816 (118.69 GB)
DFS Used: 273700913 (261.02 MB)
Non DFS Used: 95328942031 (88.78 GB)
DFS Remaining: 31842639872 (29.66 GB)
DFS Used%: 0.21%
DFS Remaining%: 24.99%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Thu Sep 10 00:01:51 BDT 2026
Last Block Report: Wed Sep 09 23:27:27 BDT 2026
Num of Blocks: 49
```

Fig4: The active DataNode and its available storage information.

Description: The screenshot shows the Hadoop DataNode report, including the active DataNode and its available storage information.

4.5 Record Count Verification

The total number of records stored in the HDFS input files was independently verified using:

```
hdfs dfs -cat /bda_final_project/input/*.txt | find /c /v ""
```

The result was: 749,361

Therefore: Total records stored in HDFS = 749,361

This verification is important because it confirms that the complete dataset was available in HDFS before performing MapReduce and Pig analysis.

Screenshot

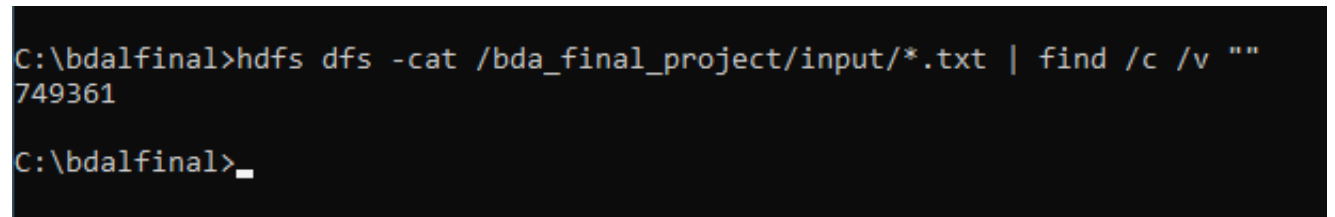
A screenshot of a terminal window with a black background and white text. The text shows a command being executed: 'C:\bdalfinal>hdfs dfs -cat /bda_final_project/input/*.txt | find /c /v ""' followed by the output '749361'. Below this, the prompt 'C:\bdalfinal>' is shown with a cursor.

Fig5: final_varification.png.

Description

The screenshot shows the final HDFS record-count verification and confirms that 749,361 records were available from the HDFS input directory.

4.6 HDFS Health and Final Verification

After uploading and checking the dataset, the HDFS input directory was verified for block health.

The final HDFS results were:

HDFS Property	Result
Dataset Files	5
Total Records	749,361
Total HDFS Blocks	6
Replication Factor	1
Missing Blocks	0
Corrupt Blocks	0
Under-replicated Blocks	0
HDFS Status	Successfully uploaded and verified

These results confirm that the dataset was successfully stored in HDFS and was ready for subsequent MapReduce and Pig processing.

5. MapReduce Analysis

Java MapReduce was used to perform category-based analytical operations on the YouTube dataset.

The project implemented three MapReduce programs:

CategoryCount.java

CategoryTotalViews.java

CategoryAverageRating.java

MapReduce follows a general processing pattern in which the Mapper extracts and transforms input data, while the Reducer aggregates values belonging to the same key. The reference material also describes Driver, Mapper, and Reducer roles for analytical jobs.

5.1 Category-wise Video Count

Objective:

Returns the total number of videos for each YouTube category.

This analysis comprises of:

1. **Driver Class:** Sets up the MapReduce job configuration, mapper and reducer classes, input/output paths, and output data types.
2. **Mapper Class:** Extracts `category` from each video record and emits **category (Text)** as key and **1 (IntWritable)** as value.
3. **Reducer Class:** Calculates the total number of videos for each category by summing the mapper values.

Code: `CategoryCount.java`

```
import java.io.IOException;
```

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class CategoryCount {
    public static class CategoryMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final IntWritable one = new IntWritable(1);
        private final Text category = new Text();
        public void map(Object key, Text value, Context context)
```

```

        throws IOException, InterruptedException {
            String line = value.toString().trim();

            if (line.isEmpty()) {
                return;
            }
            String[] fields = line.split("\\t", -1);
            // Dataset format:
            // video_id uploader age category length views rate ratings comments ...
            if (fields.length >= 4) {
                category.set(fields[3]);
                context.write(category, one);
            }
        }
    }

    public static class CategoryReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
        private final IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable> values,
            Context context)
            throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args)
        throws Exception {
        if (args.length != 2) {
            System.err.println(
                "Usage: CategoryCount <input path> <output path>"
            );
            System.exit(2);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Category Wise Video Count");
        job.setJarByClass(CategoryCount.class);
        job.setMapperClass(CategoryMapper.class);
        job.setReducerClass(CategoryReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(
            job, new Path(args[0])
        );

        FileOutputFormat.setOutputPath(
            job, new Path(args[1])
        );
    }
}

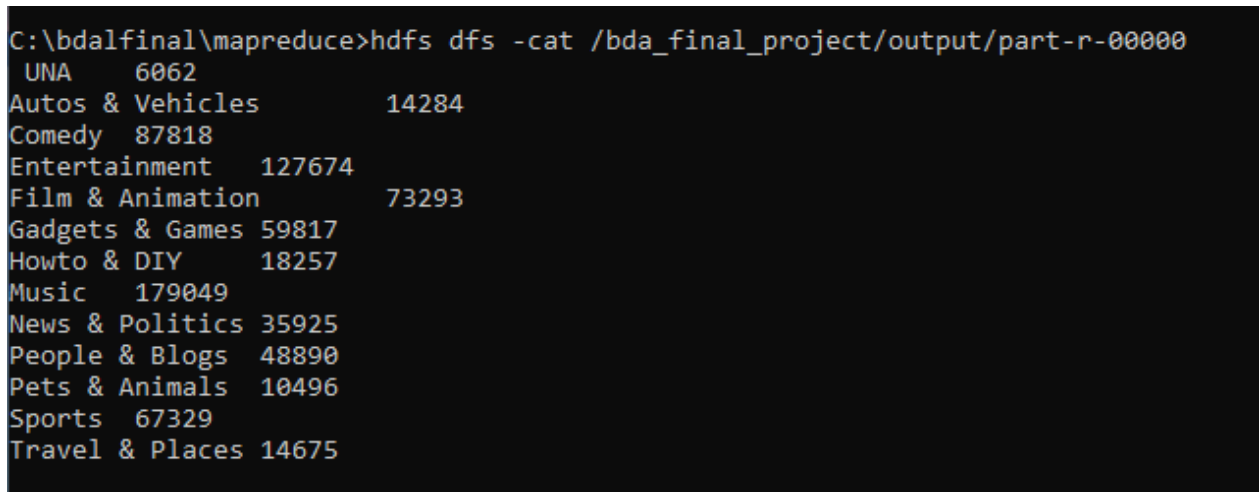
```

```

    );
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Output Snapshot:



```

C:\bdalfinal\mapreduce>hdfs dfs -cat /bda_final_project/output/part-r-00000
UNA      6062
Autos & Vehicles      14284
Comedy   87818
Entertainment  127674
Film & Animation      73293
Gadgets & Games  59817
Howto & DIY      18257
Music    179049
News & Politics  35925
People & Blogs   48890
Pets & Animals   10496
Sports    67329
Travel & Places  14675

```

Fig5:category_count_result.png here.

5.2 Category-wise Total Views

Objective:

Returns the total number of views for each YouTube video category.

This analysis comprises of:

1. **Driver Class:** Configures the MapReduce job and defines the mapper, reducer, input/output paths, and output types.
2. **Mapper Class:** Extracts `category` and `views` from each record and emits **category (Text)** as key and **views (LongWritable)** as value.
3. **Reducer Class:** Calculates the total views for each category by summing the view values.

Code: CategoryTotalViews.java

```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

```

```

public class CategoryTotalViews {
    public static class ViewsMapper
        extends Mapper<Object, Text, Text, LongWritable> {
        private final Text category = new Text();
        private final LongWritable views = new LongWritable();
        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty()) {
                return;
            }
            String[] fields = line.split("\\t", -1);
            // Dataset format:
            // video_id, uploader, age, category, length, views, ...
            if (fields.length >= 6) {
                try {
                    String cat = fields[3].trim();
                    long viewCount = Long.parseLong(fields[5].trim());

                    if (!cat.isEmpty()) {
                        category.set(cat);
                        views.set(viewCount);
                        context.write(category, views);
                    }
                } catch (NumberFormatException e) {
                    // Ignore invalid view values
                }
            }
        }
    }

    public static class ViewsReducer
        extends Reducer<Text, LongWritable, Text, LongWritable> {
        private final LongWritable result = new LongWritable();
        public void reduce(Text key, Iterable<LongWritable> values,
            Context context)
            throws IOException, InterruptedException {
            long totalViews = 0;
            for (LongWritable value : values) {
                totalViews += value.get();
            }
            result.set(totalViews);
            context.write(key, result);
        }
    }

    public static void main(String[] args)
        throws Exception {

        if (args.length != 2) {
            System.err.println(
                "Usage: CategoryTotalViews <input path> <output path>"
            );
        }
    }
}

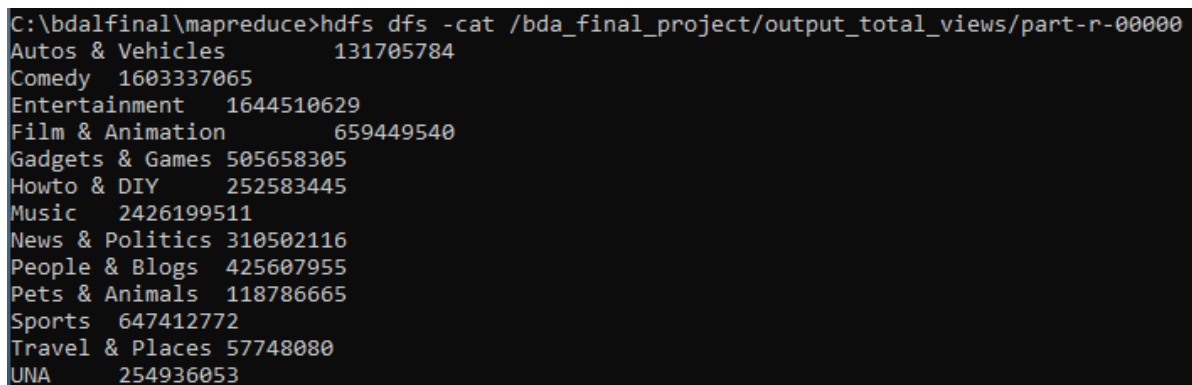
```

```

    );
    System.exit(2);
}
Configuration conf = new Configuration();
Job job = Job.getInstance(
    conf, "Category Wise Total Views"
);
job.setJarByClass(CategoryTotalViews.class);
job.setMapperClass(ViewsMapper.class);
job.setReducerClass(ViewsReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(LongWritable.class);
FileInputFormat.addInputPath(
    job, new Path(args[0])
);
FileOutputFormat.setOutputPath(
    job, new Path(args[1])
);
System.exit(
    job.waitForCompletion(true) ? 0 : 1
);
}
}

```

Output Snapshot:



```

C:\bdalfinal\mapreduce>hdfs dfs -cat /bda_final_project/output_total_views/part-r-00000
Autos & Vehicles      131705784
Comedy  1603337065
Entertainment  1644510629
Film & Animation    659449540
Gadgets & Games  505658305
Howto & DIY      252583445
Music   2426199511
News & Politics  310502116
People & Blogs   425607955
Pets & Animals  118786665
Sports   647412772
Travel & Places  57748080
UNA      254936053

```

Fig6: category_total_views_result.png.

5.3 Category-wise Average Rating

Objective:

Returns the average rating for each YouTube video category.

This analysis comprises of:

1. **Driver Class:** Sets up the MapReduce job configuration, mapper and reducer classes, input/output paths, and output types.
2. **Mapper Class:** Extracts `category` and `rate` from each video record and emits **category (Text)** as key and **rating (DoubleWritable)** as value.
3. **Reducer Class:** Calculates the total rating and number of ratings, then computes the **average rating for each category**.

Code: CategoryAverageRating.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class CategoryAverageRating {
    public static class RatingMapper
        extends Mapper<Object, Text, Text, DoubleWritable> {
        private final Text category = new Text();
        private final DoubleWritable rating = new DoubleWritable();
        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty()) {
                return;
            }
            String[] fields = line.split("\\t", -1);
            // Dataset format:
            // video_id, uploader, age, category, length, views, rate, ...

            if (fields.length >= 7) {
                try {
                    String cat = fields[3].trim();
                    double rate = Double.parseDouble(fields[6].trim());

                    if (!cat.isEmpty()) {
                        category.set(cat);
                        rating.set(rate);
                        context.write(category, rating);
                    }
                } catch (NumberFormatException e) {
                    // Ignore invalid rating values
                }
            }
        }
    }
}
```

```

    }

    public static class RatingReducer
        extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
        private final DoubleWritable result = new DoubleWritable();
        public void reduce(Text key, Iterable<DoubleWritable> values,
            Context context)
            throws IOException, InterruptedException {
            double totalRating = 0.0;
            long count = 0;
            for (DoubleWritable value : values) {
                totalRating += value.get();
                count++;
            }
            if (count > 0) {
                result.set(totalRating / count);
                context.write(key, result);
            }
        }
    }

    public static void main(String[] args)
        throws Exception {
        if (args.length != 2) {
            System.err.println(
                "Usage: CategoryAverageRating <input path> <output path>"
            );
            System.exit(2);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(
            conf, "Category Wise Average Rating"
        );
        job.setJarByClass(CategoryAverageRating.class);

        job.setMapperClass(RatingMapper.class);
        job.setReducerClass(RatingReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(
            job, new Path(args[0])
        );

        FileOutputFormat.setOutputPath(
            job, new Path(args[1])
        );

        System.exit(
            job.waitForCompletion(true) ? 0 : 1
        );
    }
}

```

Output Snapshot:

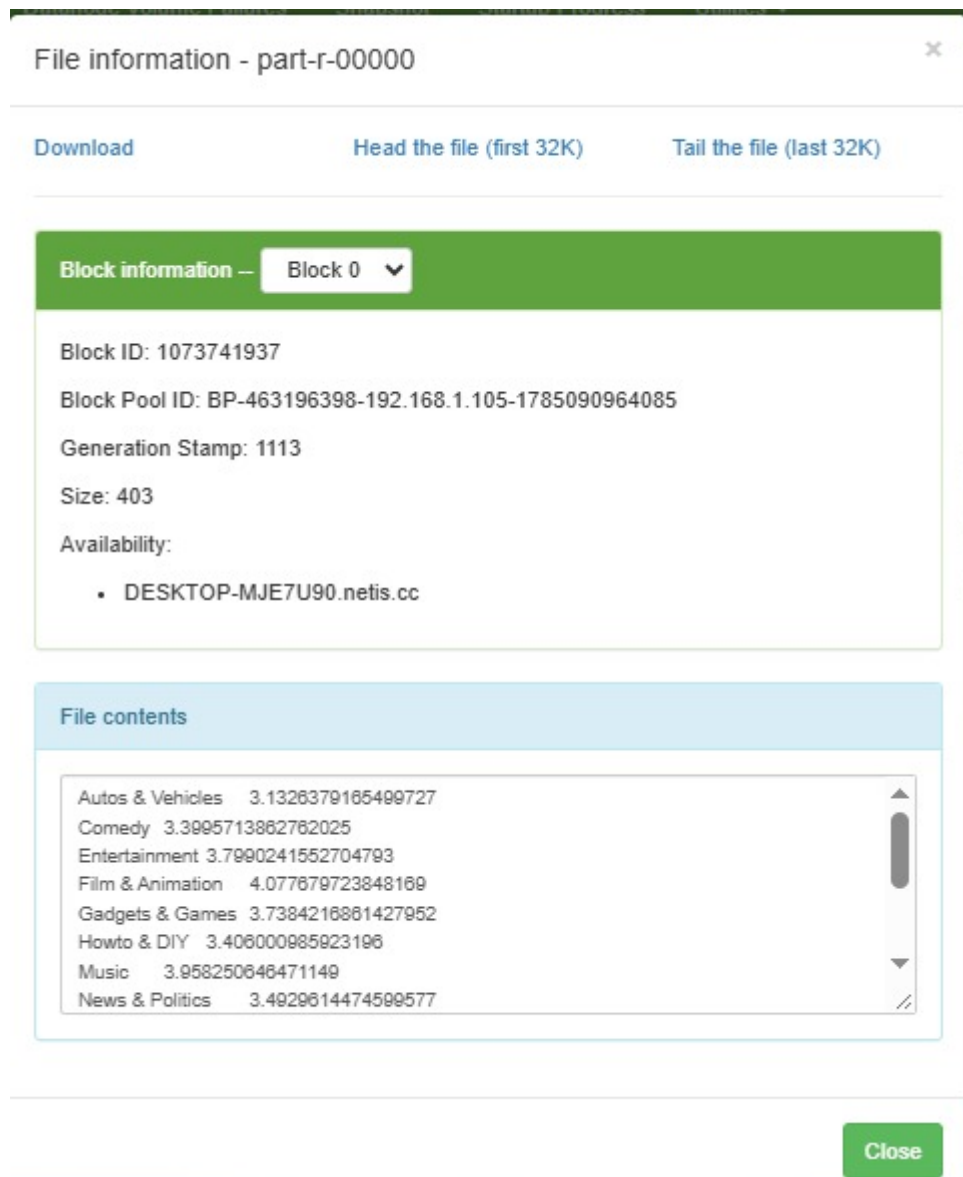


Fig7: category_average_rating_result.png.

5.4 MapReduce Analysis Summary

MapReduce Program	Purpose	Main Result
CategoryCount.java	Count videos by category	Music = 179,049
CategoryTotalViews.java	Sum views by category	Music = 2,426,199,511
CategoryAverageRating.java	Average rating by category	Film & Animation $\approx$ 4.08

The implemented operations represent grouping and summarization, which are key concepts highlighted in the reference criteria.

6. Pig Analysis

1. Total Number of Videos

Objective:

Calculates the total number of video records available in the dataset.

This analysis comprises of:

1. **LOAD:** Loads all YouTube video records from HDFS using `PigStorage('\t')`.
2. **GROUP:** Groups all records together using `GROUP data ALL`.
3. **COUNT:** Counts all records using the `COUNT()` function and returns the total number of videos.

Code Snippet: `count_data.pig`

```
data = LOAD '/bda_final_project/input/*.txt'
```

```
USING PigStorage('\t')
```

```
AS (
```

```
    video_id:chararray,
```

```
    uploader:chararray,
```

```
    age:int,
```

```
    category:chararray,
```

```
    length:int,
```

```
    views:long,
```

```
    rate:double,
```

```
    ratings:int,
```

```
    comments:int
```

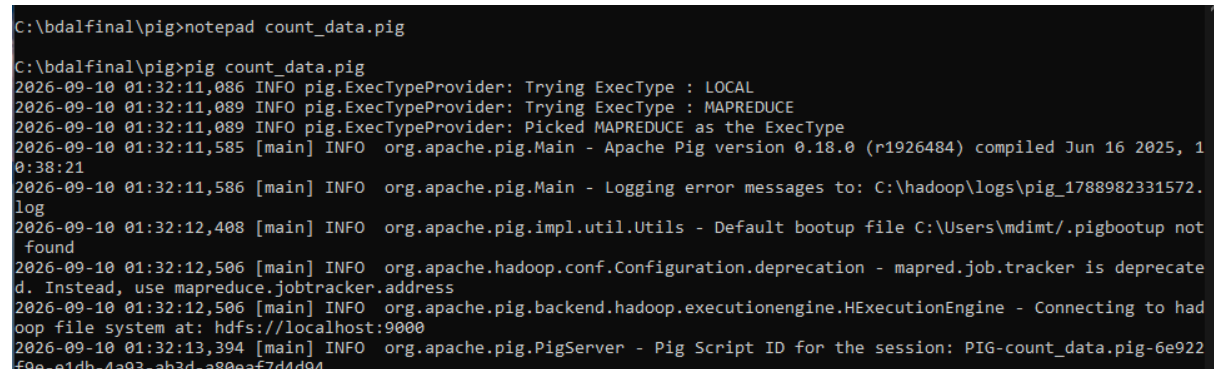
```
);
```

```
total_count = GROUP data ALL;
```

```
result = FOREACH total_count GENERATE COUNT(data);
```

```
DUMP result;
```

Output Snapshot:



```
C:\bdalfinal\pig>notepad count_data.pig
C:\bdalfinal\pig>pig count_data.pig
2026-09-10 01:32:11,086 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-10 01:32:11,089 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-10 01:32:11,089 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-10 01:32:11,585 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
2026-09-10 01:32:11,586 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788982331572.log
2026-09-10 01:32:12,408 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\mdimt/.pigbootup not found
2026-09-10 01:32:12,506 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-10 01:32:12,506 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-10 01:32:13,394 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-count_data.pig-6e922f9e-e1db-4a93-ab3d-a80eaf7d4d94
```

Fig8: pig_count_result_1.png

```

2026-09-10 01:41:08,976 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020.
Already tried 8 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS
)
2026-09-10 01:41:12,025 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020.
Already tried 9 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS
)
2026-09-10 01:41:14,173 [main] WARN org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - U
nable to retrieve job to compute warning aggregation.
2026-09-10 01:41:14,174 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - S
uccess!
2026-09-10 01:41:14,195 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - yarn.resourcemanager.system-met
rics-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
2026-09-10 01:41:14,199 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will
not generate code.
2026-09-10 01:41:14,231 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to proces
s : 1
2026-09-10 01:41:14,232 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths
to process : 1
(749361)
2026-09-10 01:41:14,415 [main] INFO org.apache.pig.Main - Pig script completed in 9 minutes, 3 seconds and 419 millise
conds (543419 ms)

```

Fig9: *pig_count_result_2.png*.

Output:

749,361 total video records

2. Category-wise Video Count

Objective:

Returns the total number of videos for each YouTube category.

This analysis comprises of:

1. **LOAD:** Loads the YouTube dataset from HDFS and defines the required fields.
2. **GROUP:** Groups the records based on the `category` field.
3. **COUNT:** Counts the number of records in each category using the `COUNT()` function.

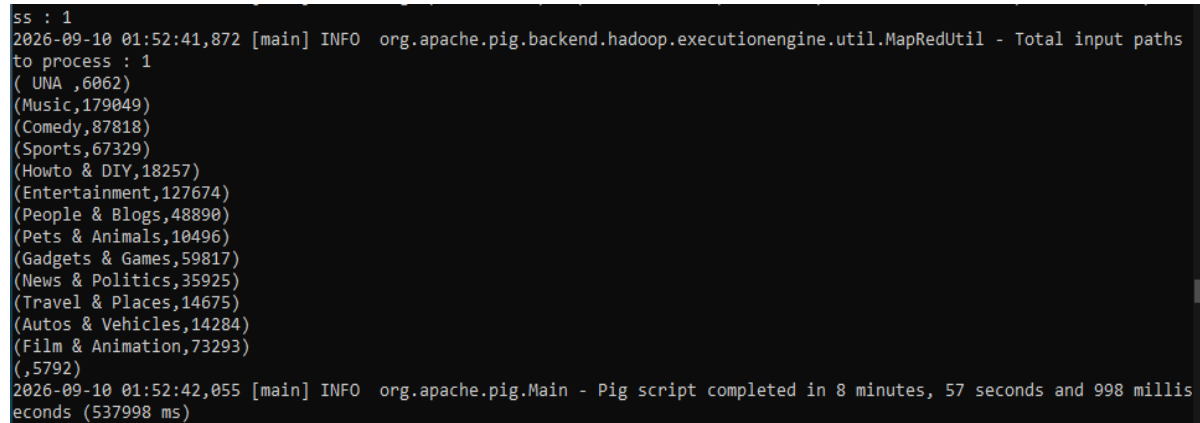
Code Snippet: `category_count.pig`

```

data = LOAD '/bda_final_project/input/*.txt'
USING PigStorage('\t')
AS (
    video_id:chararray,
    uploader:chararray,
    age:int,
    category:chararray,
    length:int,
    views:long,
    rate:double,
    ratings:int,
    comments:int
);
grouped = GROUP data BY category;
result = FOREACH grouped GENERATE
    group,
    COUNT(data);
DUMP result;

```

Output Snapshot:

A terminal window with a dark background showing the output of a Pig script. The output lists 15 categories and their corresponding view counts, followed by a completion message. The categories and counts are: UNA (6062), Music (179049), Comedy (87818), Sports (67329), Howto & DIY (18257), Entertainment (127674), People & Blogs (48890), Pets & Animals (10496), Gadgets & Games (59817), News & Politics (35925), Travel & Places (14675), Autos & Vehicles (14284), Film & Animation (73293), and an unlabeled category with 5792 views. The script completed in 8 minutes, 57 seconds, and 998 milliseconds.

```
ss : 1
2026-09-10 01:52:41,872 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths
to process : 1
( UNA ,6062)
(Music,179049)
(Comedy,87818)
(Sports,67329)
(Howto & DIY,18257)
(Entertainment,127674)
(People & Blogs,48890)
(Pets & Animals,10496)
(Gadgets & Games,59817)
(News & Politics,35925)
(Travel & Places,14675)
(Autos & Vehicles,14284)
(Film & Animation,73293)
(,5792)
2026-09-10 01:52:42,055 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds and 998 millis
econds (537998 ms)
```

Fig10: pig_category_count_result.png.

3. Category-wise Total Views

Objective:

Calculates the total number of views for each YouTube video category.

This analysis comprises of:

1. **LOAD:** Loads the dataset from HDFS with `views` defined as a numeric field.
2. **GROUP:** Groups all video records according to their `category`.
3. **SUM:** Calculates the total views for each category using the `SUM()` function.

Code Snippet: `category_total_views.pig`

```
data = LOAD '/bda_final_project/input/*.txt'
USING PigStorage('\t')
AS (
    video_id:chararray,
    uploader:chararray,
    age:int,
    category:chararray,
    length:int,
    views:long,
    rate:double,
    ratings:int,
    comments:int
);
grouped = GROUP data BY category;
result = FOREACH grouped GENERATE
    group,
    SUM(data.views);
DUMP result;
```

Output Snapshot:

```
ss : 1
2026-09-10 02:03:56,406 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths
to process : 1
(UNA,254936053)
(Music,2426199511)
(Comedy,1603337065)
(Sports,647412772)
(Howto & DIY,252583445)
(Entertainment,1644510629)
(People & Blogs,425607955)
(Pets & Animals,118786665)
(Gadgets & Games,505658305)
(News & Politics,310502116)
(Travel & Places,57748080)
(Autos & Vehicles,131705784)
(Film & Animation,659449540)
(,
)
2026-09-10 02:03:56,597 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds and 792 millis
seconds (537792 ms)
```

Fig11: pig_category_total_views_result.png

4. Category-wise Average Rating

Objective:

Calculates the average rating for each YouTube video category.

This analysis comprises of:

1. **LOAD:** Loads the dataset from HDFS and defines the `rate` field as a numerical value.
2. **GROUP:** Groups the video records based on the `category` field.
3. **AVG:** Calculates the average rating for each category using the `AVG()` function.

Code Snippet: `category_average_rating.pig`

```
data = LOAD '/bda_final_project/input/*.txt'
USING PigStorage('\t')
AS (
    video_id:chararray,
    uploader:chararray,
    age:int,
    category:chararray,
    length:int,
    views:long,
    rate:double,
    ratings:int,
    comments:int
);
grouped = GROUP data BY category;
result = FOREACH grouped GENERATE
    group,
    AVG(data.rate);
DUMP result;
```

Output Snapshot:

```
not generate code.
2026-09-10 02:14:19,318 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input fi
ss : 1
2026-09-10 02:14:19,319 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total
to process : 1
( UNA ,3.945923787528868)
(Music,3.9582506464710625)
(Comedy,3.39957138627618)
(Sports,3.5106722214796053)
(Howto & DIY,3.4060009859232054)
(Entertainment,3.799024155270472)
(People & Blogs,3.00290857025975)
(Pets & Animals,2.9413967225609725)
(Gadgets & Games,3.738421686142738)
(News & Politics,3.492961447459976)
(Travel & Places,2.6897989778534828)
(Autos & Vehicles,3.13263791654998)
(Film & Animation,4.077679723848124)
(,)
2026-09-10 02:14:19,526 [main] INFO org.apache.pig.Main - Pig script completed in 8 minutes, 57 seconds an
econds (537486 ms)
```

Fig12: Insert pig_category_average_rating_result.png

7. Result

The project successfully completed the Big Data processing workflow using HDFS, MapReduce, and Apache Pig.

7.1 Overall Result

- Total Records: 749,361
- HDFS Blocks: 6
- Replication Factor: 1
- Missing/Corrupt/Under-replicated Blocks: 0

7.2 MapReduce Result

Three MapReduce analyses were successfully completed:

- Highest Video Count: Music — 179,049 videos
- Highest Total Views: Music — 2,426,199,511 views
- Highest Average Rating: Film & Animation — ≈ 4.08

7.3 Pig Result

The same analyses were performed using Apache Pig. The results were consistent with MapReduce:

Analysis	MapReduce	Pig
Total Records	749,361	749,361
Music Video Count	179,049	179,049
Music Total Views	2,426,199,511	2,426,199,511
Highest Average Rating	Film & Animation ≈ 4.08	Film & Animation ≈ 4.08

7.4 Key Findings

- Music was the most represented and most viewed category.
- Film & Animation had the highest average rating.
- 5,792 records contained missing/empty category values.
- Consistent MapReduce and Pig results confirm the correctness of the analysis.

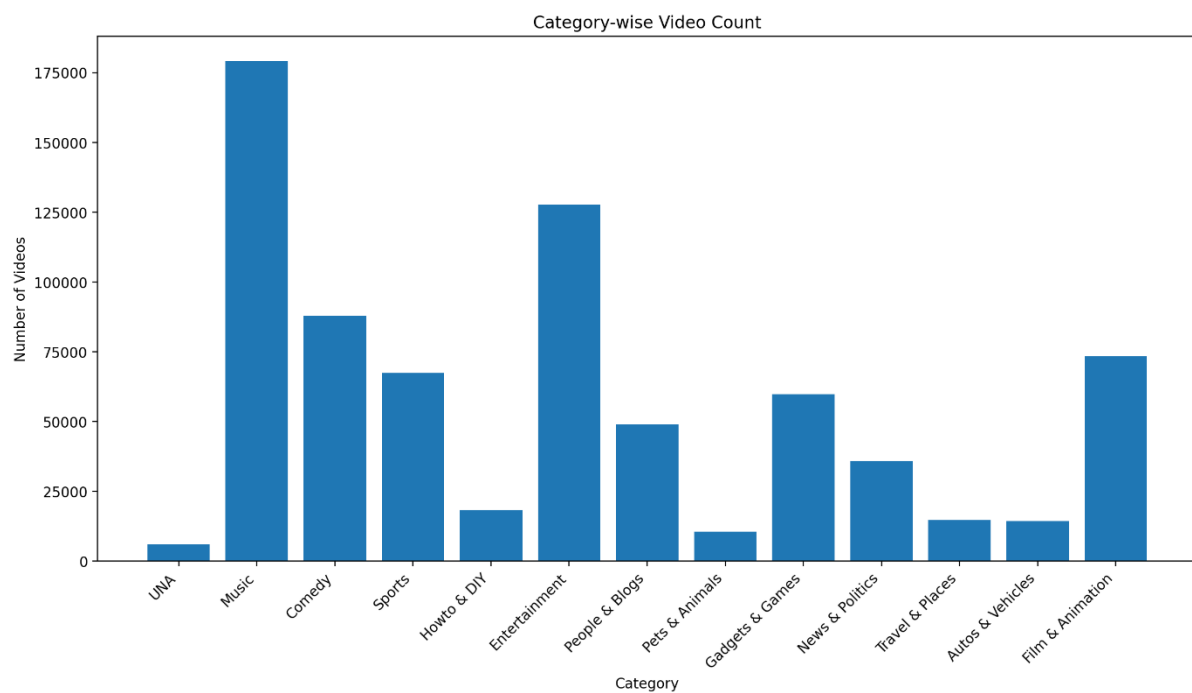
7.5 Visualization Results

Three visualizations were generated from the analytical results.

Category-wise Video Count

Fig13: category_video_count.png.

Description:



This visualization compares the number of YouTube videos across categories. Music has the highest number of videos, followed by Entertainment and Comedy.

Category-wise Total Views

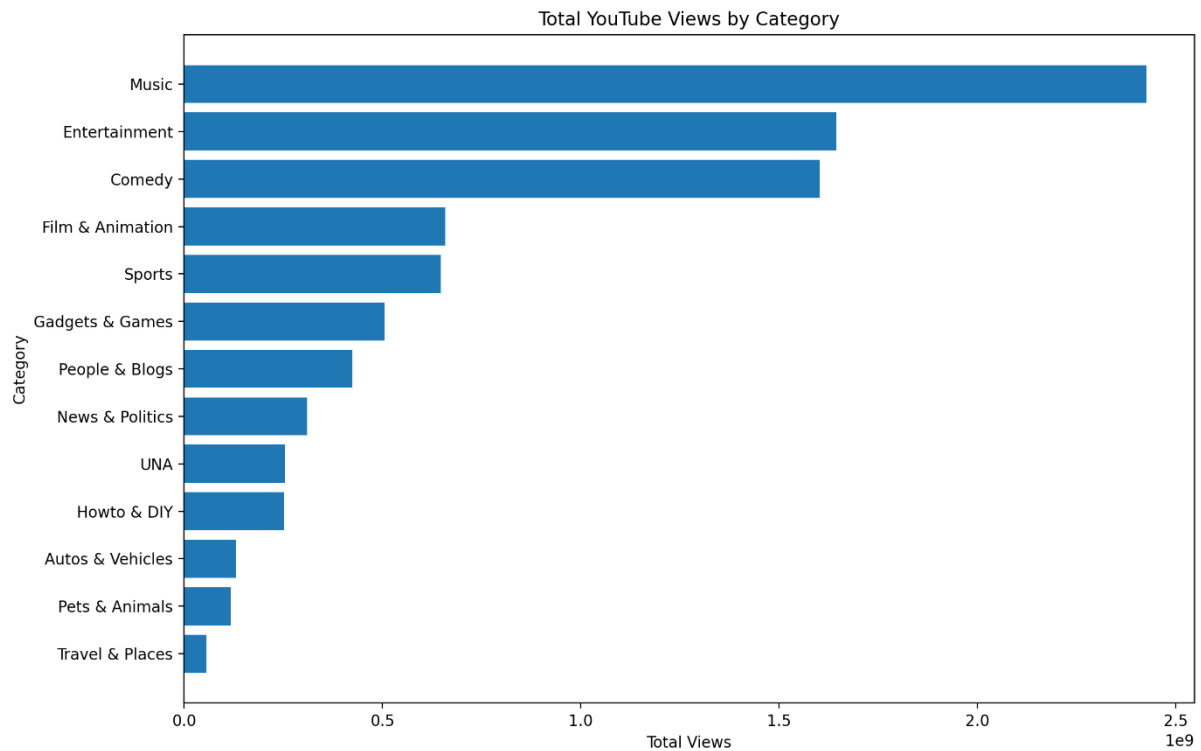


Fig14:category_total_views.png.

Description:

This visualization compares the total number of views received by each category. Music has the highest total view count.

Category-wise Average Rating

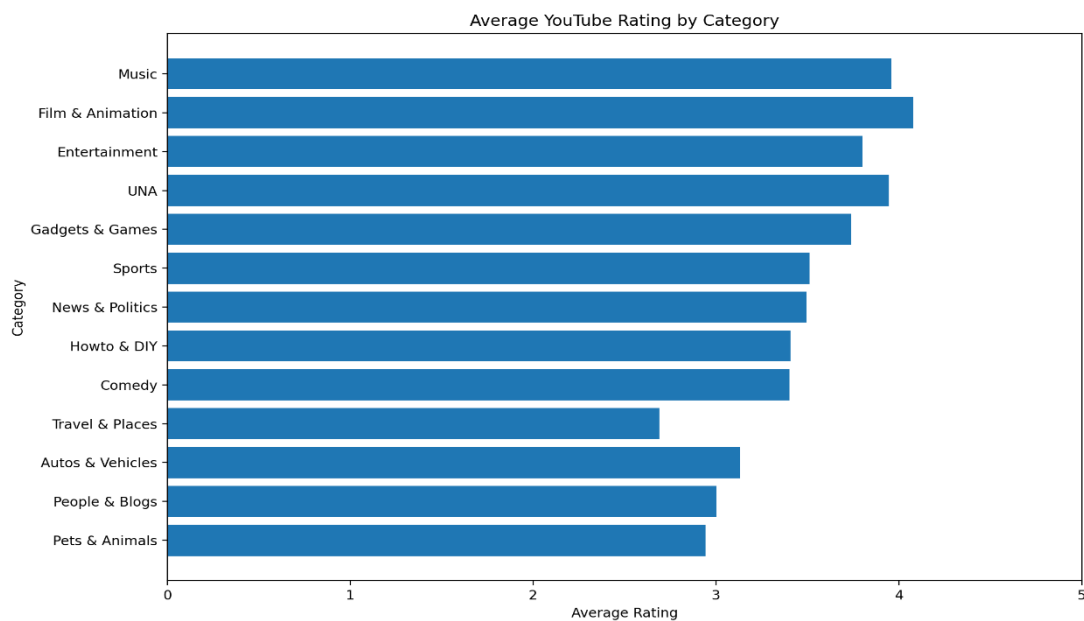


Fig15: category_average_rating.png.

Description:

This visualization compares the average rating of videos across categories. Film & Animation achieves the highest average rating.

8. Conclusion

This project successfully demonstrated the use of Hadoop HDFS, Java MapReduce, and Apache Pig for analyzing a large YouTube dataset containing 749,361 records. HDFS was used to store and manage the dataset, while MapReduce and Pig were used for category-wise video count, total views, and average rating analysis. The results from both technologies were consistent, validating the correctness of the analysis. The findings showed that Music had the highest number of videos and total views, while Film & Animation had the highest average rating of approximately 4.08. The analysis also identified 5,792 records with missing category values, highlighting a data-quality issue. Overall, the project provided practical experience in distributed storage, large-scale data processing, aggregation, and result validation using Hadoop technologies

The complete implementation, Java MapReduce source code, Pig scripts, HDFS commands, screenshots, and visualizations are available in the GitHub repository:

GitHub:

https://github.com/Imtiaz-1132/BDAL_Final_Project.git